

CS 4032 – Web Programming



Contact Details

- Dr. Tajwar Mehmood
- Email: tajwar.mehmood@nu.edu.pk
- Office:
 - Room# C-204B
- Contact Hours
 - Monday- Thursday: 2:30PM to 3:30PM



Course Objectives

- Understanding of modern web application development technologies
- Develop and design Web interfaces using latest UI frameworks
- Understand the best web development practices being followed in the industry
- Training on industry-oriented web frameworks
- Utilize the incredible power of web technologies
 - Develop web based of MEAN, MERN, and advance technologies.



Course is really different mini-courses

Frontend Technologies

HTML5, CSS3, JavaScript
Angular, ReactJS

Backend Technologies

NodeJS, ExpressJS,
MongoDB

.net Technologies
Asp.net MVC

Web Service Technologies

Amazon AWS
Google Cloud
Azure Services



Target Students

- Want to pursue good career as a **Full Stack Web Developer/Web Designer**
- Want to learn cutting-edge Web development technologies
- Want to get hands-on practice on industry-oriented web frameworks
- Feel Comfortable in coding (e.g., C++, Java, Python...)
 - Has built, or could build, a single-user application
- Ready to take coding challenges



Contents and Organization

Topics to be covered:			
List of Topics	No. of Weeks	Contact Hours	CLO(s)
Introduction to Web Development Front-end vs Back-end Development	1	3	1,2,7,12
HTML, HTML5, CSS, CSS3, Bootstrap, tailwind	2	6	1,2,3,5,8
JavaScript fundamentals , JQuery	1	3	1,2,3,5,8
AJAX, JQUERY, FETCH, AXIOS	1	3	1,2,3,5,8
State management and Data Bindings	1	3	1,3,4,12
MongoDB	1	3	5,12,6
Introduction to Node	1	3	1,2,3,5
Introduction to Express	1	3	1,2,3,5
Programming with Angular and Typescript	2	6	1,3,4,5,11,12
Introduction to React Asynchronous JavaScript	2	6	1,2,3,5
Introduction to Vue	1	3	1,2,3,5
Advance topics- Serverless , GraphQL	1	6	1,2,3,4, 5,12
Deployment and Web Programming practices and Demos	1	3	1,6,7,9,10,11
Total	16	48	



Contents and Organization

COURSE CONTENTS (Lab/ Practical):

Weeks	Contents/Topics	** Courseware Events (MM/ IT Lab/Case Study/ Assignment/ Presentation etc.)	Comments (if any)
Week-01	Introduction to web technologies		
Week-02	Developing web pages using HTML and HTML5		
Week-03	Designing web pages using CSS and CSS3	Quiz 1	
Week-04	Application of Java Script for Web Application development	Assignment 1	
Week-05	State Management Techniques and Data binding	Quiz 2	
Week-06	Introduction to Angular	Assignment 2	
Week-07	Programming with Angular and TypeScript	Quiz 3	
Week-08	Angular MVC/MVVM architecture	Assignment 3	
Week-09	Developing database using MongoDB		
Week-10	Introduction to NodeJS	Project	
Week-11	Programming with ExpressJS and NodeJS	Quiz 4	
Week-12	Programming with ReactJS	Assignment 4	
Week-13	Hands on experience on MEAN and MERN Stack	Quiz 5	
Week-14	Asp.net MVC	Assignment 5	
Week-15	Hands on practice on Serverless Stack Development		



Tentative Marks Distribution

Assessment Item	Number	Weight (%)
Quizzes/Tasks	≥ 8	15
Assignments	≥ 5	20
Sessional Exam	2	15
Project	1	15
Final Exam	1	35

Grading policy: *Absolute grading scheme*



Class Policies and Guidelines

Attendance policy:

- **Will be taken at the start of the class.** Students *appearing late in the class after the attendance* will be marked **“Absent”**
- **80%** attendance is compulsory

Plagiarism policy:

- Plagiarism in **midterm/final** exam may result in **F grade** in the course.
- Plagiarism in an assignment items (**assignments, quizzes & project**) will result in zero marks in the whole assignments items category. If fore mentioned act is repeated more than once the instructor can refer a case to the Department Disciplinary Committee (the maximum punishment can be award of '**F' grade** in that course.)



Class Policies and Guidelines

Course retake policy:

- **Midterm/final exam retake**
 - The examination assessment and retake committee decide the exam retake/pretake cases.
- **Assignments/quizzes retake**
 - There will be **no retake** of any assignment or quiz.



Class Policies and Guidelines

Quiz Policy:

- Class Tasks would be considered as QUIZ.
- Submission of Class Task would be required.
- Demo of Class Tasks are also required to get marked.
- **Re-take of Demo would NOT be taken.**
- **On Missing Demo**
 - **Evaluation would be based on Code submission**
 - **40% deduction would be applied.**



Class Policies and Guidelines

Tasks, Assignments or project submission policy:

- Use the following rules for your submissions
 - Combine all files in one **zip** file
 - Name the zip file as **ROLLNO_NAME_SECTION.zip**
 - Submit the zip file on **github-Classroom** before the deadline
 - Your Github Account should be with your fullName
- **On Missing Assignment/Project Demo:**
 - **Evaluation would be based on Code submission**
 - **40% deduction would be applied.**



Class Policies and Guidelines

Late Submission policy:

- Late for First hour – 10 % deduction
- Late for 12 hours – 20% deduction
- Late for 24 hours – 30% deduction
- No submission after 24 hours



Assignments and Projects

- Where **~80%** of your learning will take place
- Posted to **Google Classroom** and **Slate**
- All **tasks, assignments** will be individual
- **Project** will be in group (max. 2-3 students)
- Program must work, compile errors / runtime errors lose all correctness points
- Copying solution code or giving code to someone else is **CHEATING -> F** in the course.



Class Policies and Guidelines

- **Don'ts**

- Use of cell phones
- Discussion with fellows during class
- Early leave
- Frequent movements In-out of class

- **Do's**

- Bring your own laptops along with chargers- **Mandatory**
- Be interactive, ask questions
- Participate in the lecture especially during hands-on practice/Tasks



Textbooks and Reference Material

- **Web Application Architecture Principles, protocols and practices** by Leon Shklar and Richard Rosen
- **Learning JavaScript**, 3rd Edition by Todd Brown



Google Classroom

- Invite link
 - <https://classroom.google.com/c/NTg1Mzg3NTUyNzc2?cjc=h5bxpfa>
- Class code
 - h5bxpfa
- Lectures, reference material, announcements



Introduction to Web



National University
Of Computer and Emerging Sciences

CORE TECHNOLOGIES



CSS PRE-PROCESSORS



JS FRAMEWORKS & LIBRARIES



TASK RUNNERS



DEPENDENCY & PACKAGE MANAGERS



CSS FRAMEWORKS



MISC TECH



TESTING

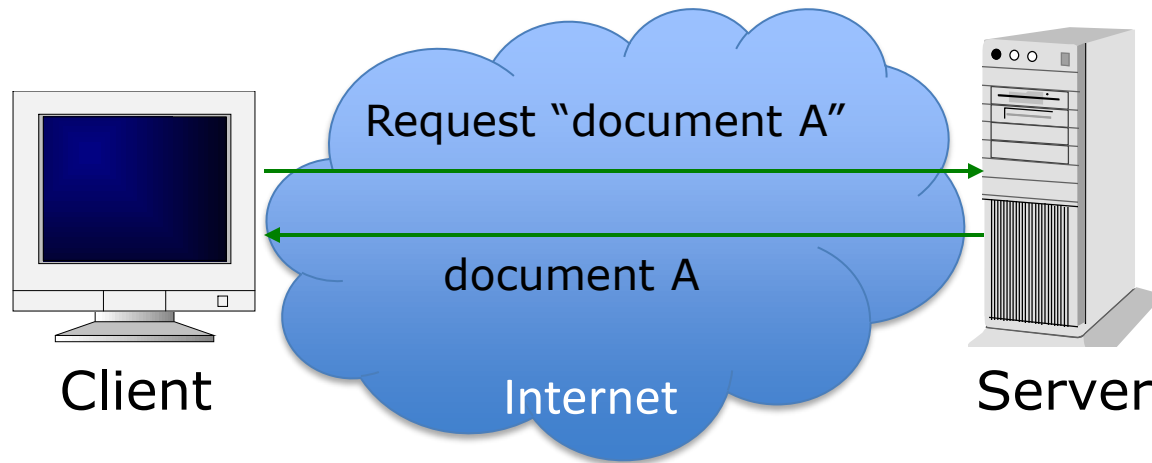


ADDITIONAL SKILLS

Accessibility
Agile
GitHub & Git
Search Engine Optimization (SEO)

Web Essentials

- **Client:** web browsers, used to surf the Web
- **Server systems:** used to supply information to these browsers
- **Computer networks:** used to support the browser-server communication



Internet v.s. Web

The Internet: a inter-connected computer networks, linked by wires, cables, wireless connections, etc.

Web: a collection of interconnected documents and other resources.

The world wide web (**WWW**) is accessible via the Internet, as are many other services including email, file sharing, etc.

Through communication
protocols



How does the
Internet
Work?

A **communication protocol** is a
specification of how
communication between two
computers will be carried out

IP (Internet Protocol): defines the packets that carry blocks of data from one node to another

TCP (Transmission Control Protocol) and **UDP** (User Datagram Protocol): the protocols by which one host sends data to another.

Other application protocols: **DNS** (Domain Name Service), **SMTP** (Simple Mail Transfer Protocol), and **FTP** (File Transfer Protocol)



The Internet Protocol (IP)

- A key element of IP is **IP address**, a 32-bit number
- The Internet authorities assign ranges of numbers to different organizations
- **IP** is responsible for moving packet of data from node to node
- A **packet** contains information such as the **data** to be transferred, the **source** and **destination** IP addresses, etc.
- Packets are sent through different local network through gateways
- A **checksum** is created to ensure the correctness of the data; corrupted packets are discarded
- IP-based communication is unreliable

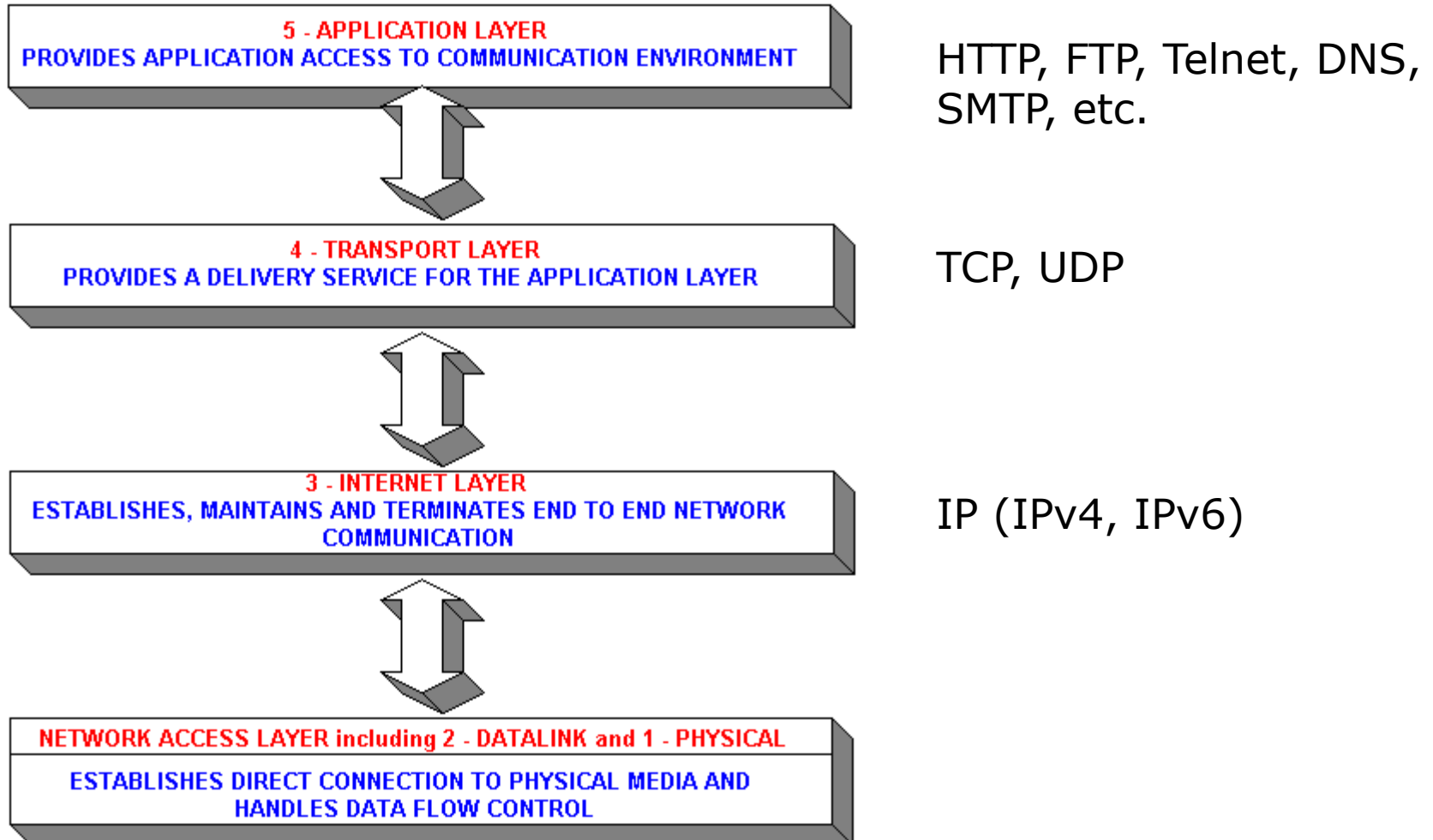


The Transmission Control Protocol (TCP)

- TCP is a higher-level protocol that extends IP to provide additional functionality
 - **reliable** communication
 - **two-way** (full duplex) communication
- TCP adds support to detect errors or lost data and to trigger **retransmission** until the data is correctly and completely received
- Connection
- Acknowledgment



TCP/IP Protocol Suites



The World Wide Web (WWW)

- **WWW** is a system of interlinked, hypertext documents that runs over the Internet
- Two types of software:
 - **Client**: a system that wishes to access the information provided by servers must run client software (e.g., web browser)
 - **Server**: an internet-connected computer that wishes to provide information to others must run server software
 - Client and server applications communicate over the Internet by following a protocol built on top of TCP/IP – **HyperText Transport Protocol (HTTP)**



Basics of the WWW

- **Hypertext**: a format of information which allows one to move from one part of a document to another or from one document to another through hyperlinks
- **Uniform Resource Locator (URL)**: unique identifiers used to locate a particular resource on the network
- **Markup language**: defines the structure and content of hypertext documents

Web Client: Browser

- Makes **HTTP requests** on behalf of the user
 - Reformat the URL entered as a valid **HTTP request**
 - Use **DNS** to convert server's host name to appropriate IP address
 - Establish a **TCP connection using the IP address**
 - Send HTTP request over the connection and wait for server's response
 - Display the document contained in the response
 - If the document is not a plain-text document but instead is written in **HTML**, this involves rendering the document (positioning text, graphics, creating table borders, using appropriate fonts, etc.)

Web Servers

- Main functionalities:
 - Server waits for connect requests
 - When a connection request is received, the server creates a new process to handle this connection
 - The new process establishes the **TCP** connection and waits for **HTTP requests**
 - The new process invokes software that maps the requested **URL** to a resource on the server
 - If the resource is a file, creates an **HTTP response** that contains the file in the body of the response message
 - If the resource is a program, runs the program, and returns the output

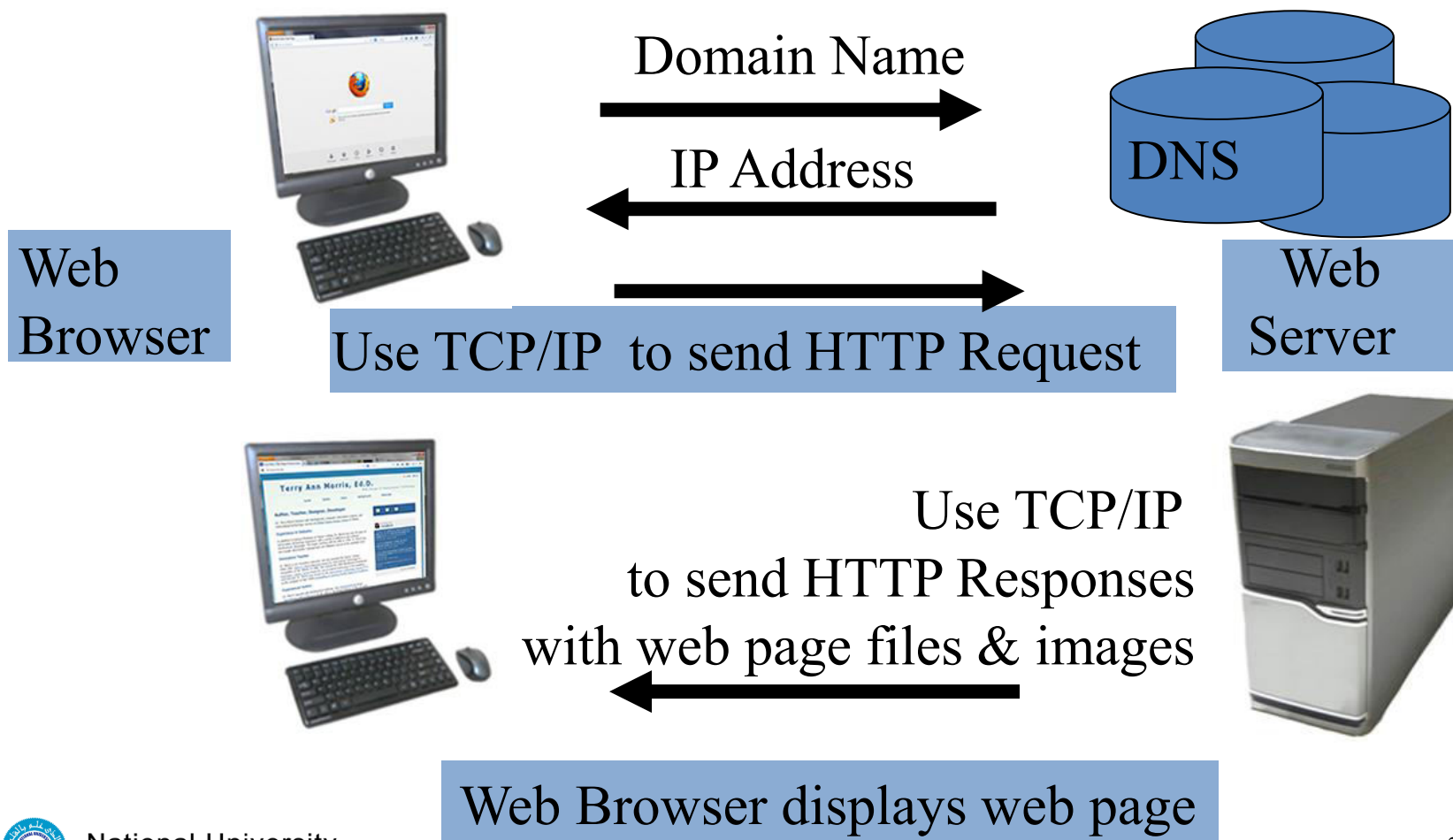
Domain Name

- Locates an organization or other entity on the Internet
- Domain Name System (DNS)
 - Divides the Internet into logical groups and understandable names
 - Associates unique computer IP Addresses with the text-based domain names you type into a web browser
 - Browser: <http://google.com>
 - IP Address: 173.194.116.72



Domain Name System

- The Domain Name System (DNS) associates Domain Names with IP addresses.



Top-Level Domain (TLD) Name

- A **top-level domain (TLD)** identifies the right-most part of the domain name.
- Examples of generic TLDs:
.com, .org, .net, .mil, .gov, .edu, .int, .aero, .asia, .cat, .jobs, .name, .biz, .mobi, .museum, .info, .coop, .post, .pro, .tel, .travel



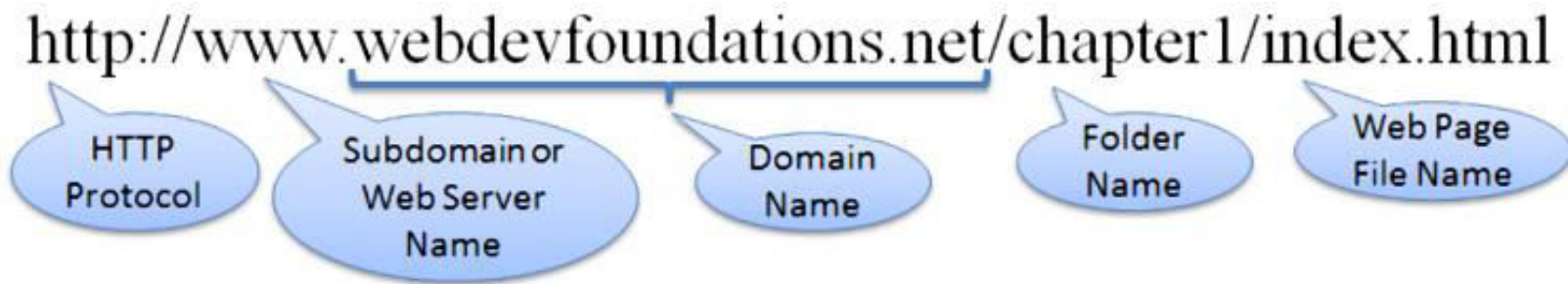
County Code TLDs

- Two character codes originally intended to indicate the geographical location (country) of the web site.
- In practice, it is fairly easy to obtain a domain name with a country code TLD that is not local to the registrant.
- Examples:
 - .tv, .ws, .au, .jp, .uk
 - See <http://www.iana.org/cctld/cctld-whois.htm>



Uniform Resource Identifier

- **URI** – Uniform Resource Identifier
 - identifies a resource on the Internet
- **URL** – Uniform Resource Locator
 - a type of URI which represents the network location of a resource such as a web page, a graphic file, or an MP3 file.



Static Web: HTML/XHTML, CSS

- **HTML** stands for **H**yper**T**ext **M**arkup **L**anguage
 - It is a text file containing small markup tags that tell the Web browser how to display the page
- **XHTML** stands for e**X**tensible **H**yperText **M**arkup **L**anguage
 - It is identical to HTML 4.01
 - It is a stricter and cleaner version of HTML
 - E.g., `<!DOCTYPE>`, `<html>`, `<head>`, and `<body>` are mandatory
- **CSS** stands for **C**ascading **S**tyle **S**heets
 - It defines how to display HTML elements

Static web limitations

- What is the drawback to simple document model?
 - Static
 - Assume that documents are created before they are requested
- What are examples of information that might be part of web documents that may not be known before they are requested?

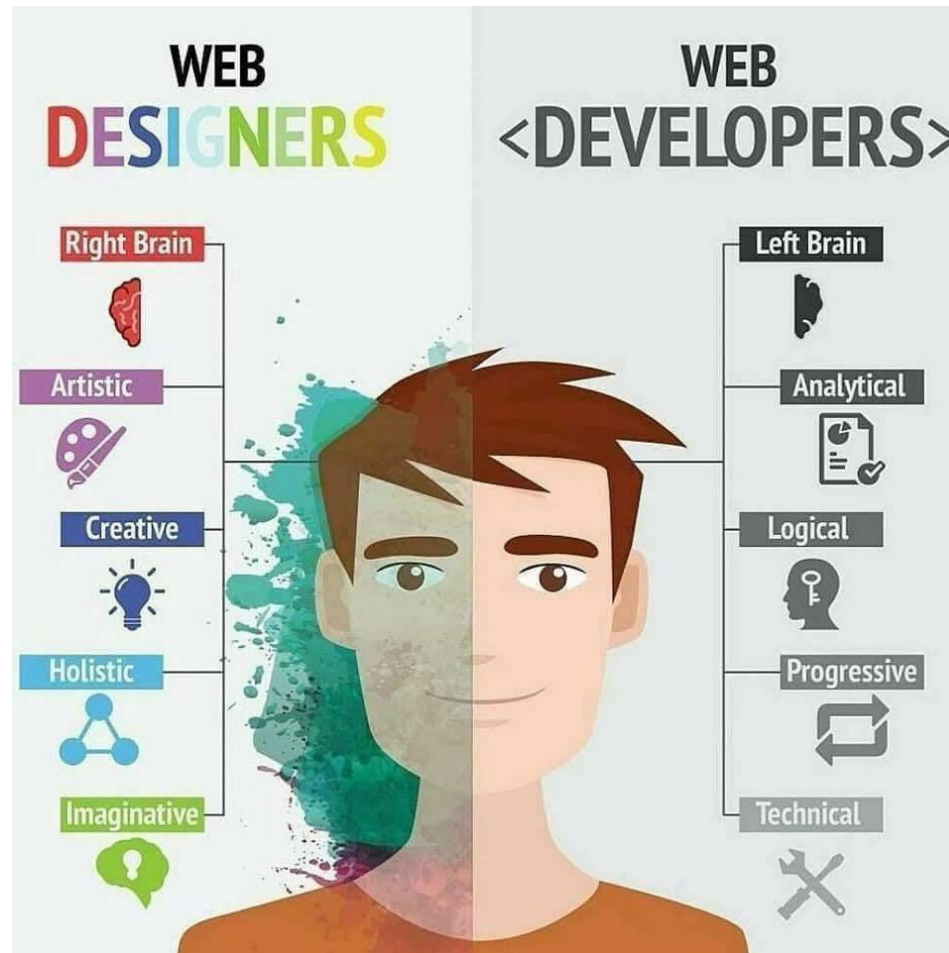
Client-Side Programming

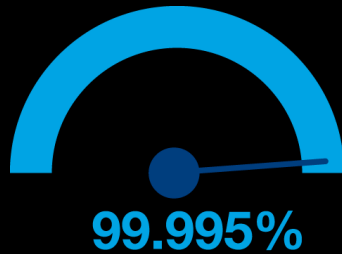
- **Scripting language**: a lightweight programming language
- Browser scripting: **JavaScript**
 - Designed to add interactivity to HTML pages
 - Usually embedded into HTML pages
 - What can a JavaScript Do?
 - Put dynamic text into an HTML page
 - React to events
 - Read and write HTML elements
 - Validate data before it is submitted to a server
 - Create cookies
 - ...

Server-Side Programming

- The requests cause the response to be generated
- Server scripting:
 - **ASP.Net MVC**: Microsoft product, uses .Net framework (*.asp)
 - CGI/Perl: Common Gate Way Interface (*.pl, *.cgi)
 - PHP: Open source, strong database support (*.php)
 - Java via JavaServer Pages (*.jsp)
 - ...

Web Development vs Designing



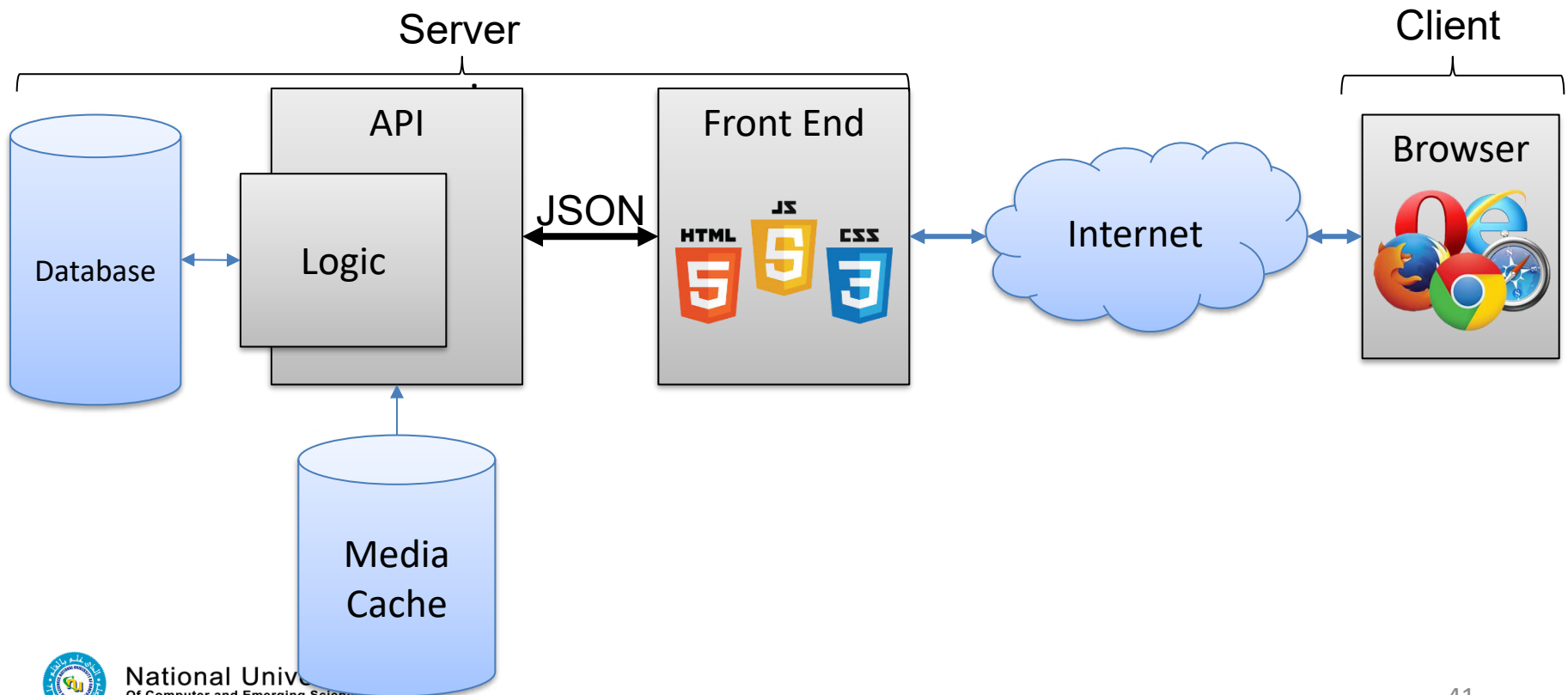


Principles of Web Design

- Availability
- Performance
- Reliability
- Scalability
- Manageability
- Cost

Web Applications

- UI (Front End (DOM, Framework))
- Request Layer (Web API)
- Back End (Database, Logic)



FRONTEND DEVELOPMENT

HTML



CSS



JS



Front End Languages

- HTML/CSS
 - JavaScript
 - Java (applets)
-
- What is the most popular?
 - Answer: **JavaScript/HTML/CSS** is the only real option for front-end native languages and is basically the standard. But there are many variations on JavaScript that are used.

2009

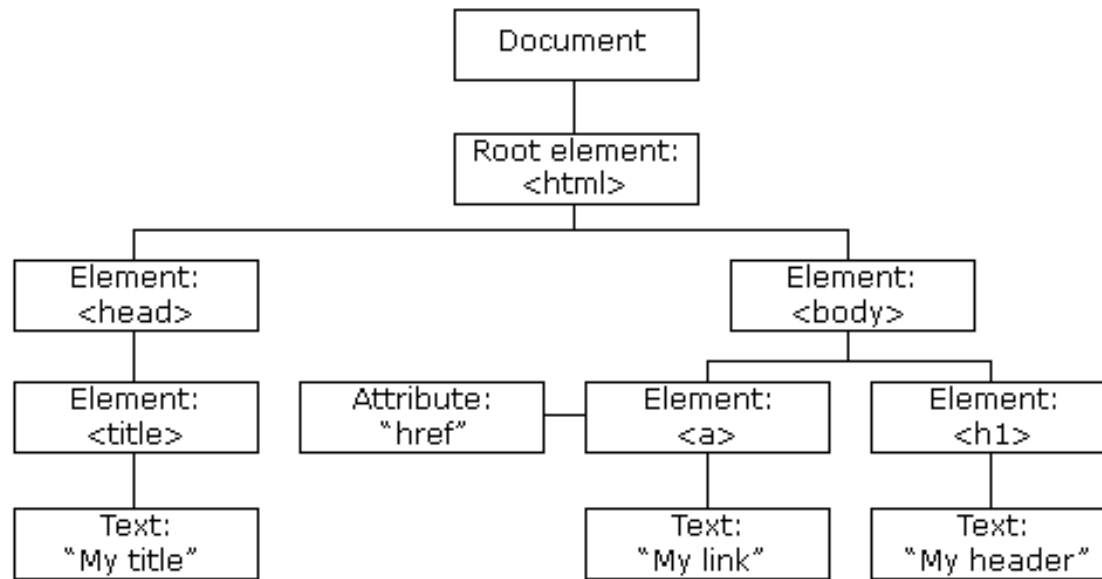


Today

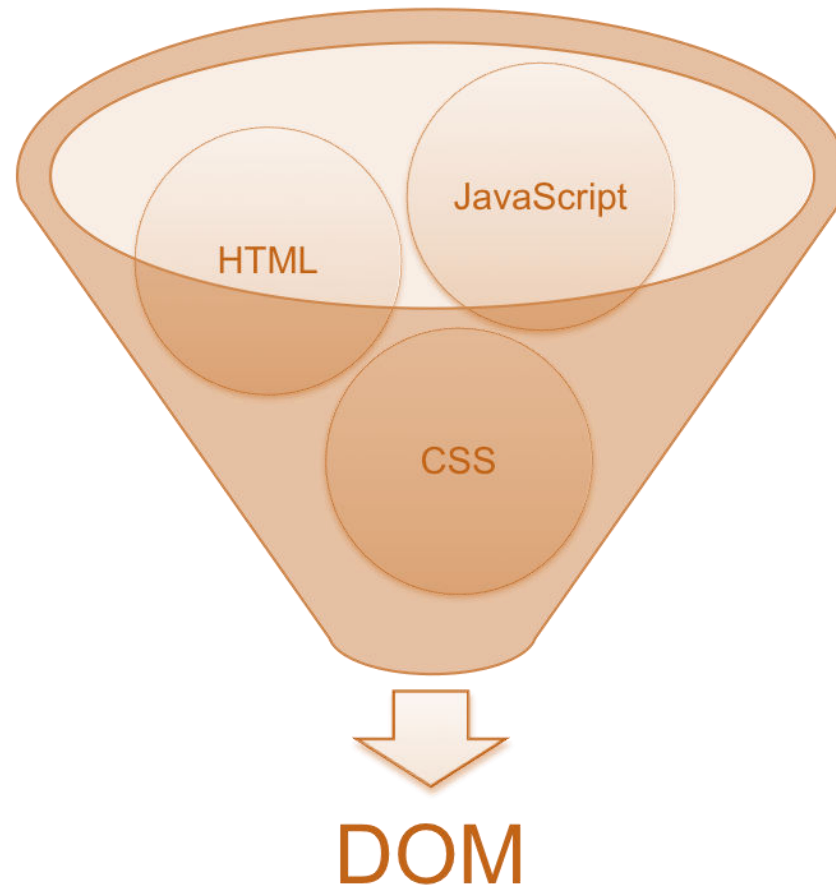


DOM (Document Object Model)

- **Document Object Model** makes every addressable item in a web application an Object that can be manipulated for color, transparency, position, sound and behaviors.
- Every **HTML Tag** is a **DOM object**



DOM (Document Object Model)



Three representations of same page

The image shows a screenshot of a web browser displaying the Google homepage. The address bar shows `http://www.google.com/`. The main content area features the Google logo and a green button labeled "Visual". Below the logo, there are navigation links: "Web", "Images", "Groups", "News", "Froogle", "Local", and "more »".

Overlaid on the bottom of the browser window are two green buttons: "DOM" on the left and "HTML" on the right. The "DOM" button is positioned over the DOM tree view, which shows the document's structure with elements like `<HTML>`, `<HEAD>`, `<BODY>`, `Links (15)`, `Images/Flashes(2/0)`, `Forms (1)`, and `Scripts (3)`. The "HTML" button is positioned over the HTML (Rendered) view, which displays the source code of the page, including the `<HTML>`, `<HEAD>`, `<TITLE>`, `<META>`, `<STYLE>`, and `<TD>` tags.

Software Architectural Pattern and Framework

- Software Architectural Pattern
 - An architectural pattern expresses a fundamental structural organization schema for software systems
 - It provides a set of predefined subsystems, their responsibilities, and includes rules and guidelines for organizing the relationships between them
- Software Framework
 - A platform for developing software applications
 - It provides a foundation on which software developers can build programs for a specific platform
 - May include predefined classes and functions or APIs that can be used to process input, manage hardware/software components, etc.

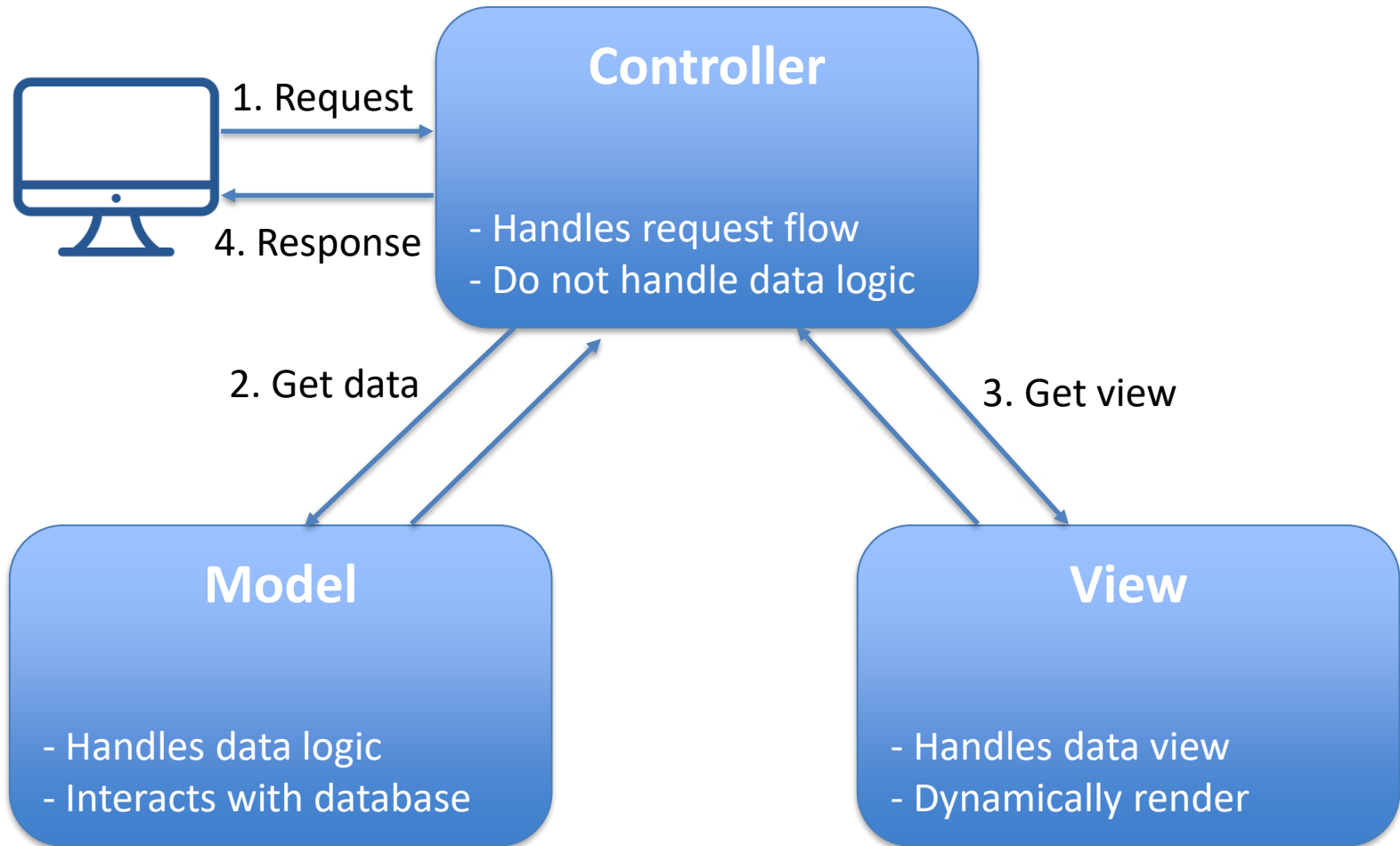


MVC (Model View Controller)

- Software architectural design pattern
- Designed in 1970
- Originally developed for desktop applications
 - Most frequently used for web applications
- Separates application functionality
- Promotes modular programming



MVC Overview

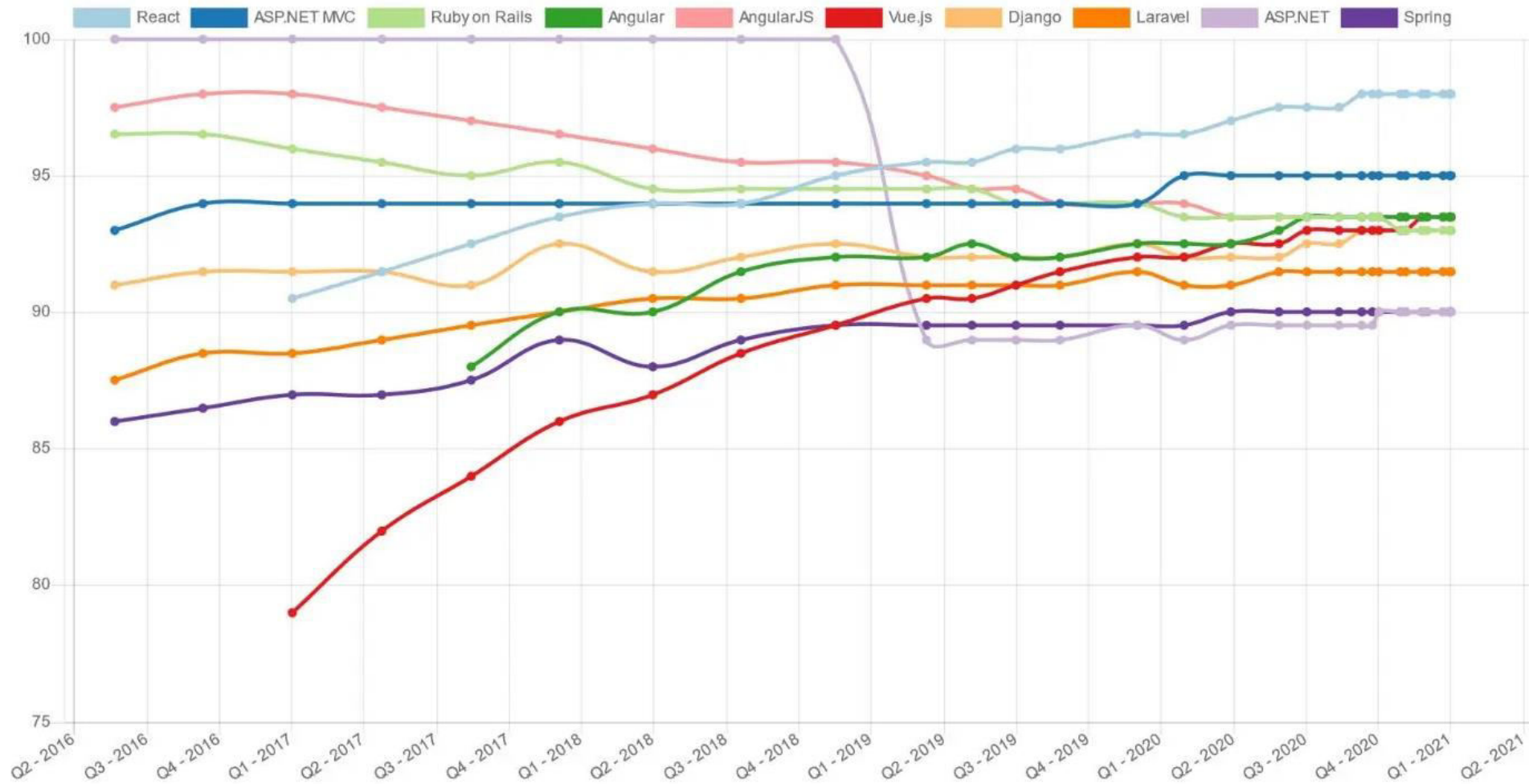


Web Frameworks That Use MVC

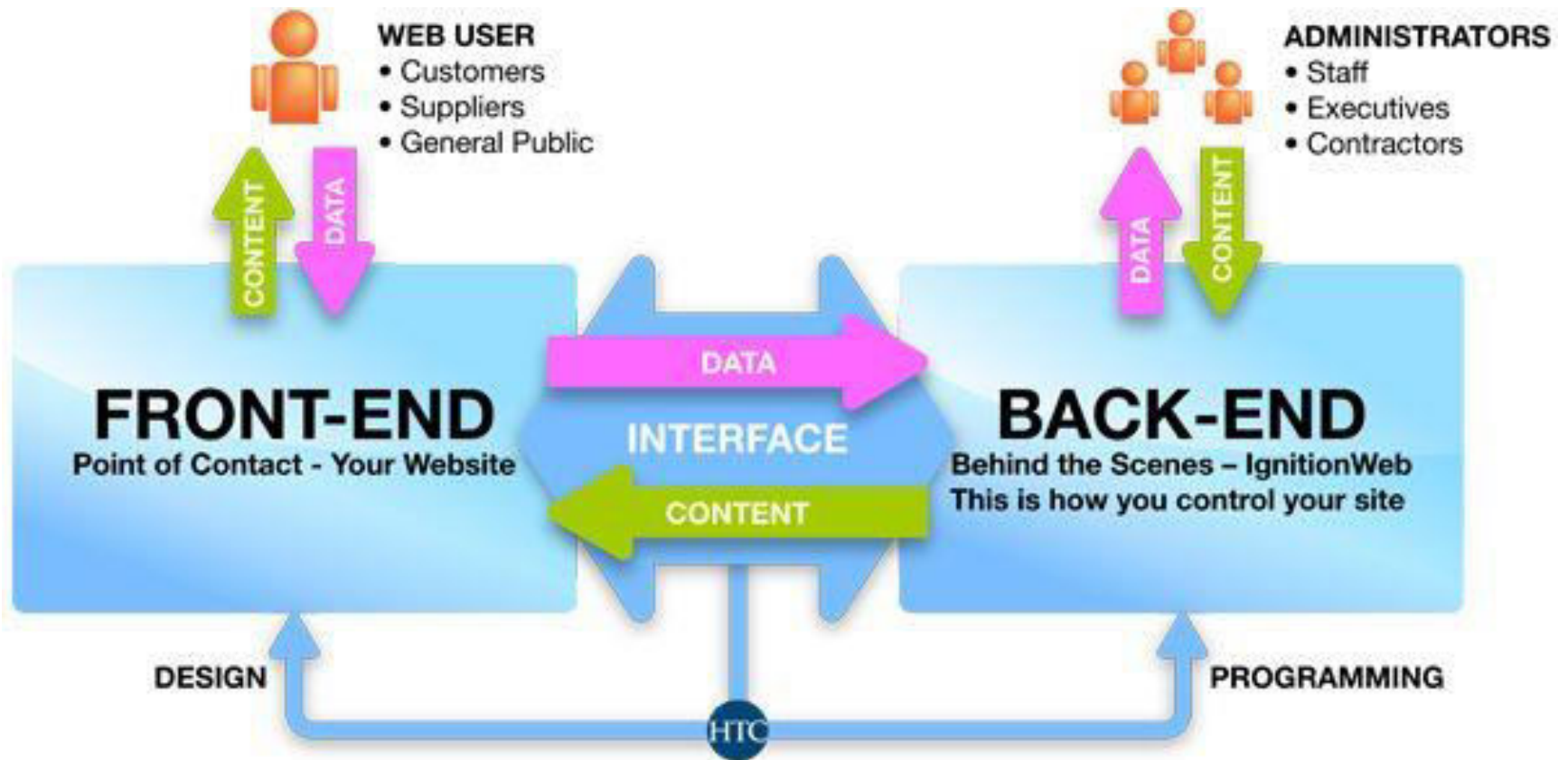
- Spring (Java)
- ASP.NET MVC (C#)
- Ruby on Rails (Ruby)
- Laravel (PHP)
- Django (Python)
- Flask(Python)
- AngularJS
- React (JS)
- Ember (JS)
- Express (JS)
- Vue (JS)
- Polymer (JS)



Popular frameworks



BACKEND DEVELOPMENT



What is a Backend?

- All of the awesome that runs your application.
- **Web API**
 - Connection layer between the frontend and backend
 - Connected through API calls (**POST, GET, PUT, etc.**)
 - Transmit Content from the Backend to the Frontend commonly in **JSON Blobs**
- Service Architecture that drives everything (Where all the logic is)

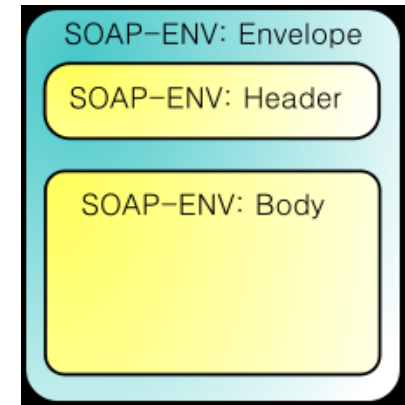


Web APIs

- **Application Programming Interface (API)**
 - Serves as an interface between different software programs and facilitates their interaction
 - A particular set of rules and specifications that a software program can follow to access and make use of the services and resources provided by another particular software program
- **Web API**
 - The intermediate layer between front end and back-end systems
 - A “must have” if your APIs will be consumed by third-party services
 - Typically a defined set of **HTTP request** messages expressed in **SOAP** or **REST** along with a definition of the structure of response messages, typically expressed in **JSON** or **XML**.



SOAP



- Simple Object Access Protocol (SOAP)
- SOAP Introduction:
 - <http://msdn.microsoft.com/en-us/library/ms995800.aspx>
- A SOAP message is an ordinary XML document containing the following elements:
 - An **Envelope element** that identifies the XML document as a SOAP message
 - A **Header element** that contains header information
 - contains application-specific information about the SOAP message
 - optional
 - must be the first child element of the Envelope element
 - A **Body element** that contains call and response information
 - A **Fault element** containing errors and status information

REST

- Representational State Transfer (REST)
 - Use HTTP method to invoke remote services (not XML)
- The response of remote service can be in XML/JSON or any textual format
- Benefits:
 - Easy to develop
 - Easy to debug (with standard browser)
 - Leverage existing web application infrastructure

```
GET /articles/23 HTTP/1.1  
Accept: text/html, application/xhtml
```

```
HTTP/1.1 200 (OK)  
Content-Type: text/html
```



Server Responses

- Really Simple Syndication (RSS, Atom)
 - XML-based standard
 - Designed for news-oriented websites to “Push” content to readers
 - Excellent to monitor new content from websites
- XML or JavaScript Object Notation (JSON)
 - Lightweight data-interchange format
 - Human readable and writable and also machine friendly
 - Wide support from most languages (Java, C, C#, PHP, Ruby, Python...)

```
1 {  
2   "firstname": "Hassan",  
3   "lastname": "Sartaj"  
4 }
```

Json: 48 bytes

```
1 <person>  
2   <first-name>Hassan</first-name>  
3   <last-name>Sartaj</last-name>  
4 </person>
```

XML: 83 bytes

Class Exercise- Developer Console

- **The 1st rule of web development:** Always keep the Developer Console open on your web
- Follow the link
https://fullstackopen.com/en/part0/fundamentals_of_web_apps#traditional-web-applications
- Open this example app
- <https://studies.cs.helsinki.fi/exampleapp/notes>
- <https://studies.cs.helsinki.fi/exampleapp/>



Home Task

- Make a Github login
- Download/clone “Task0-Welcome” from this path
 - <https://web.stanford.edu/class/archive/cs/cs193x/cs193x.1176/home%2F0-welcome>
 - Do the Task0(ungraded)
- Upload and publish your profile landing page on github



Video Lecture

- https://www.youtube.com/watch?v=f_mFUq-rHfk



Thank you!



National University
Of Computer and Emerging Sciences